GenAbu: A modular and reproducible Nextflow workflow for RNA-seq quality control and gene abundance quantification

Computational Workflows for Biomedical Data, 2026

Matthias Lang, Laura Fahrer

Abstract

RNA sequencing (RNA-seq) is widely used to investigate gene expression, but the reliability of its results depends on reproducible data processing, transparent workflows, and appropriate quality control. This project describes the development of GenAbu, a modular Nextflow workflow designed to generate gene-level abundance tables from bulk RNA-seq data. The workflow follows principles and development practices promoted by nf-core, a community-driven framework for reproducible bioinformatics pipelines. GenAbu supports three input routes: user-provided FASTQ files, sequencing data retrieved using Sequence Read Archive (SRA) accession codes, and accession information extracted from scientific publications. Its processing steps include input validation, sequence quality control with Falco, adapter and quality trimming with Cutadapt, transcript quantification with Salmon, gene-level abundance calculation, and report aggregation with MultiQC. Development emphasized automated testing, version pinning, and modularity to improve reliability and reproducibility. Remaining limitations include restricted configurability of some analysis parameters and the absence of fully integrated gene-name mapping, which is planned as a future extension to improve the interpretability of generated results.

1. Introduction

Modern high-throughput sequencing technologies can process DNA as well as RNA molecules and generate large amount of sequencing data. RNA-seq is a next-generation sequencing technique used to profile the transcriptome, detect RNA variants and quantify gene-level transcript abundance. However, the analysis of raw sequencing reads is complex and requires many subsequent steps performed by different tools. Running these tools manually through the command line can lead to inconsistent settings, undocumented parameters and difficulties reproducing previous analysis [1].

Computational pipelines address these problems by connecting individual tools into an automated workflow. They define the order of the processing steps, manage the flow of data between them and reduce the need for manual intervention. Workflow management systems such as Nextflow make it possible to execute these steps in a structured and reproducible way [2].

1.1 FAIR Principles

The FAIR principles are guidelines for making digital objects Findable, Accessible, Interoperable and Reusable. They were introduced in 2016 to improve the management and reuse of scientific data and resources. Applying these principles can also support reproducibility by improving the availability of data, metadata and information about the methods used [3].

Table 1. Meaning of the FAIR principles [3].

Principle	Description
Findable:	Data and metadata should be easy to find for both humans and computers. This can be achieved through persistent identifiers, searchable metadata or indexing in a searchable resource.
Accessible:	Data should be retrievable through standardised communication protocols. If the access to the data is restricted, there should be an standardized authentication and autorisation process available. Additionally, metadata should be accessible, even if the data is no longer available.
Interoperable:	Data should use standardised formats, vocabularies and identifiers so that they can be combined and exchanged between different systems. This enables different tools and applications to work with the same data.
Reusable:	Data and resources should be sufficiently documented to allow others to understand and reuse them. This includes information about their origin, the methods used to generate them, and clear licensing and usage conditions.

1.2 Nextflow and nf-core

Nextflow is a workflow management system designed to support the development of reproducible and portable computational workflows. It connects individual processes through channels, which manage the flow of data between workflow components. Nextflow also supports parallel execution and can be used with container technologies to provide consistent software environments [2].

nf-core is a community-driven project that builds on Nextflow to provide standardised and well-maintained bioinformatics pipelines. It offers reusable modules, pipeline templates, documentation standards and testing practices to improve the reliability and reproducibility of computational analyses [4].

GenAbu is a Nextflow workflow for RNA-seq preprocessing and gene abundance quantification. It adopts selected nf-core development practices, including a modular structure and automated testing. The aim is to provide a structured workflow for

processing sequencing data while improving maintainability and reproducibility. The following sections describe the implementation of GenAbu and discuss how its design relates to reproducibility and the FAIR principles.

2. Methods

GenAbu was implemented using Nextflow and a modular project structure. The main workflow is defined in main.nf, which includes local modules and subworkflows. Individual processing tasks are separated from the main workflow, allowing components to be maintained, modified and tested independently. The workflow uses containerised software environments to improve portability and reproducibility by reducing dependencies on software installed on the host system.

2.1 Pipeline Architecture

GenAbu supports three different types of input. User-provided FastQ files can be processed directly, whereas SRA accession codes are used to retrieve raw sequencing data from the corresponding publicly-available archive. As a third option a scientific publication can be provided to extract SRA accession codes directly, if available.

The main analysis workflow consists of three main steps, the preprocessing, quantification and reporting of the result. Preprocessing includes quality control of the raw and trimmed reads using Falco and adapter trimming with Cutadapt. Salmon is then used for transcript quantification based on pseudo-alignment. The resulting transcript-level estimates are summarized into gene-level abundance scores using the gene_abundance tool. Finally, the workflow outputs gene-level abundance tables and a MultiQC report summarising the quality control results form different tools.

The modular architecture of the pipeline combines established bioinformatics tools with components developed specifically for GenAbu. Tools such as Salmon [5] and Cutadapt [6] provide established analysis functionality, while custom modules and subworkflows handle input validation, accession extraction and resolution, data routing and gene-level abundance summarisation.

2.2 Automated Testing and Continuous Integration

To detect errors early in development and maintain a stable main branch, GenAbu uses nf-test and follows the principles of Continuous Integration (CI). nf-test is a testing framework for Nextflow workflows and enables to test all components of a pipeline, including end-to-end testing and process-level testing [7]. GenAbu includes automated workflow tests by replacing the actual processes with stub tests of predefined stub behavior. This allows to verify the functionality of the workflow, the connection of processes and the expected output structures without running the actual tools.

Additionally, a small representative test dataset was used for integration testing. This allowed to check the execution and output of the actual tools on relatively low execution time and computational resources. The combination of both approaches helps distinguish errors arising from the workflow logic from problems during data processing.

CI complements this testing strategy by automatically executing defined checks whenever changes are introduced to the code repository. This enables potential errors to be detected before changes are merged into the main branch, keeping the main branch stable and functional.

2.3 Reproducibility and Computational Environment

GenAbu was implemented using Docker for containerised execution of the individual processes. It provides the software environments required by the tools and thus supports to reduce the influence of different host environments [8]. The used container version of each tool is included in the documentation.

The source code is maintained in a public GitHub repository, allowing changes to the workflow to be tracked (C0ldCl0ud/GenAbu: Lightweight nf-pipelien for bulk RNA-sequencing data analysis).

3. Results

3.1 Input Handling

Input handling consists of several modules (see Table 2.1) and subworkflows (see Table 2.2). The workflow supports two types of input files: CSV samplesheets containing either FASTQ file paths or SRA accession codes and PDF documents of scientific publications.

The initial component, the input_router, identifies the input type based on the provided file format, distinguishing between CSV files and PDF documents. It then directs the input to the appropriate downstream module for further processing. Unused channels remain empty, allowing the workflow to handle all input types through a common entry point. For CSV inputs, the input_validator checks the samplesheet structure and prepares the data for further processing.

For publication-based input, the paper_accession subworkflow converts the PDF into text and extracts SRA accession codes. The accession_resolver then uses ffq to retrieve available metadata, including the scientific name, library strategy, library layout and FastQ download paths. Resolved accessions are saved in a reusable samplesheet, while unresolved accessions are recorded separately. In the subsequent sra_input subworkflow, the sequencing data are downloaded and converted into FastQ files for downstream analysis.

Table 2.1. Modules of GenAbu input section with corresponding description

Module	Description
accession_extractor	Extracts sequencing data accession codes from the text of a scientific publication and saves them in a TSV file.
accession_resolver	Resolves extracted accession codes using ffq and generates structured metadata, including run, sample and study accessions, scientific name, library strategy, library layout and available FASTQ download paths. It also creates a reusable samplesheet and separates resolved from unresolved accessions.
pdf_to_text	Converts a scientific publication in PDF format into a UTF-8 encoded text file, which can be processed by downstream modules.
sra_prefetch	Downloads sequencing data for a given SRA accession using the SRA Toolkit's prefetch command.

Table 2.2. Subworkflows of GenAbu input section with corresponding description

Subworkflows	Description
input_router	Identifies whether the provided input is a CSV samplesheet or a PDF document and directs the file to the appropriate downstream workflow or rejects unsupported or invalid input files.
input_validator	Detects whether a CSV samplesheet contains SRA accessions or FASTQ file paths. It validates the required columns and uses the appropriate schema to parse the samplesheet and prepare the input for downstream processing.
paper_accessions	Coordinates the publication-processing steps by converting the PDF to text, extracting accession codes and resolving them into structured metadata. It outputs resolved and unresolved accessions, a reusable samplesheet and SRA run information for further processing.
sra_input	Processes SRA-based input by downloading the sequencing data with sra_prefetch and converting it into compressed FASTQ files with fasterq-dump. It also determines whether the resulting reads are single-end or paired-end.

3.2 Analysis Workflow

The preprocessing part of the pipeline consists of an initial quality control step, followed by adapter trimming and a second QC check. To asses the quality of the raw and the trimmed reads, Falco is used. Falco is a high-speed emulation of the FastQC tool and can save computational resources by approximately running three times faster on average. It can generate quality control metrics like per-base sequence quality and sequence composition [9]. Cutadapt is used to remove unwanted sequences such as adapter sequences and to trim reads according to a quality threshold [6]. The QC and Cutadapt reports are subsequently collected by the MultiQC tool to generate a summary report instead of requiring to inspect the report of each module separately [10].

After preprocessing, the trimmed reads are passed to Salmon to quantify transcript abundance. Salmon can map reads ultra-fast through a pseudo-alignment procedure using a pre-built index derived from a reference transcriptome. This approach avoids the need for genome-alignment and enables fast and accurate quantification [5].

Gene-level abundance is determined by summarising transcript-level estimates using gene annotation information. This allows the results to be interpreted at the gene level rather than only at the transcript level possibly including several different isoforms per gene.

4. Discussion

Concepts such as Continuous Integration are fundamental to modern software development, particularly in larger, established projects. However, practical experience with these concepts is often limited in university courses, where assignments are typically divided into small, largely independent tasks. Developing this pipeline therefore provided a valuable opportunity for us to experience such a workflow firsthand.

In retrospect, adopting these practices introduced considerable friction, some of which initially seemed unnecessary. Nevertheless, several of the challenges encountered during development proved to be valuable learning experiences. For example, an earlier version of the CI configuration used latest-stable for Nextflow and ubuntu-latest for the operating system. Following a Nextflow update, tests on the main branch began to fail despite having passed previously. This demonstrated the importance of pinning software and environment versions to ensure reproducible execution.

As the number of test cases increased, testing also became increasingly time-consuming, even for minor changes or modifications unrelated to the pipeline's functionality, such as updating the README or generating figures. This was particularly frustrating when small errors required another complete test run. To address this, we adopted a more targeted approach during local development, executing individual tests for the components affected by a change before running the full test suite. Despite the

additional effort, the testing strategy contributed to a more stable pipeline and reduced the risk of introducing unintended changes in behavior.

A limitation of the current pipeline is the restricted configurability of some analysis tools. While several parameters, such as adapter sequences, quality thresholds, and minimum read lengths, are exposed to the user, other settings remain fixed. For example, Salmon uses a k-mer length of 31 during index generation. Under different experimental conditions, particularly when processing shorter reads, a smaller k-mer length might be more appropriate.

Increasing the configurability of the pipeline would therefore be a desirable improvement. However, such changes would require careful consideration of existing tests and potential effects on current behavior. For instance, changes to the k-mer length would need to be reflected in the reference caching mechanism to ensure that compatible indices are selected.

A further planned improvement is the integration of automated gene-name mapping into the pipeline. Currently, gene-level count and abundance tables may contain identifiers that are not immediately interpretable without additional annotation. The proposed extension aims to associate these identifiers with readable gene names, primarily using the NCBI gene annotation database and, where possible, information from the reference GTF file as a fallback. This would facilitate downstream analyses and improve the interpretability of the generated results. Although an initial implementation has been developed in a separate feature branch, it could not be fully completed, tested, and integrated into the main branch within the project timeframe. Completing and validating this functionality would therefore be a valuable next step in the further development of the pipeline.

Another useful improvement would be the introduction of a run-specific configuration summary alongside `software_versions.yml`, documenting all effective pipeline parameters, including both user-defined and fixed settings. This would improve traceability and reproducibility while preserving the pipeline's current ease of use.

The code was developed with support from ChatGPT, following a three-step, test-driven approach. First, tests were defined to specify the expected behavior of individual components. Second, the corresponding implementations were developed to satisfy these tests. Finally, a separate AI agent was tasked with reviewing the code against Nextflow and nf-core coding guidelines, with particular attention to readability, modularity, and maintainability. Human supervision remained essential for defining requirements, resolving ambiguities, reviewing generated code, and ensuring that the implementation remained within the intended project scope.

References

1 Kämpf, C., Specht, M., Scholz, A. et al. uap: reproducible and robust HTS data analysis. BMC Bioinformatics 20, 664 (2019). https://doi.org/10.1186/s12859-019-3219-1

2 Di Tommaso, P., Chatzou, M., Floden, E. et al. Nextflow enables reproducible computational workflows. Nat Biotechnol 35, 316–319 (2017). https://doi.org/10.1038/nbt.3820

3 Wilkinson, M., Dumontier, M., Aalbersberg, I. et al. The FAIR Guiding Principles for scientific data management and stewardship. Sci Data 3, 160018 (2016). https://doi.org/10.1038/sdata.2016.18

4 Ewels, P.A., Peltzer, A., Fillinger, S. et al. The nf-core framework for community-curated bioinformatics pipelines. Nat Biotechnol 38, 276–278 (2020). https://doi.org/10.1038/s41587-020-0439-x

5 Patro, R., Duggal, G., Love, M. et al. Salmon provides fast and bias-aware quantification of transcript expression. Nat Methods 14, 417–419 (2017). https://doi.org/10.1038/nmeth.4197

6 Martin, M. Cutadapt removes adapter sequences from high-throughput sequencing reads. EMBnet. journal 17.1 (2011): 10-12. https://doi.org/10.14806/ej.17.1.200

7 Forer, L., Schönherr, S., Improving the reliability, quality, and maintainability of bioinformatics pipelines with nf-test, GigaScience, Volume 14, 2025, giaf130, https://doi.org/10.1093/gigascience/giaf130

8 Moreau, D., Wiebels, K. & Boettiger, C. Containers for computational reproducibility. Nat Rev Methods Primers 3, 50 (2023). https://doi.org/10.1038/s43586-023-00236-9

9 de Sena Brandine G and Smith AD. Falco: high-speed FastQC emulation for quality control of sequencing data [version 2; peer review: 2 approved]. F1000Research 2021, 8:1874 https://doi.org/10.12688/f1000research.21142.2

10 Philip Ewels, Måns Magnusson, Sverker Lundin, Max Käller, MultiQC: summarize analysis results for multiple tools and samples in a single report, Bioinformatics, Volume 32, Issue 19, October 2016, Pages 3047–3048, https://doi.org/10.1093/bioinformatics/btw354